

Process APIs

2025 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 5

2.0.0

We Could Create Processes from Scratch

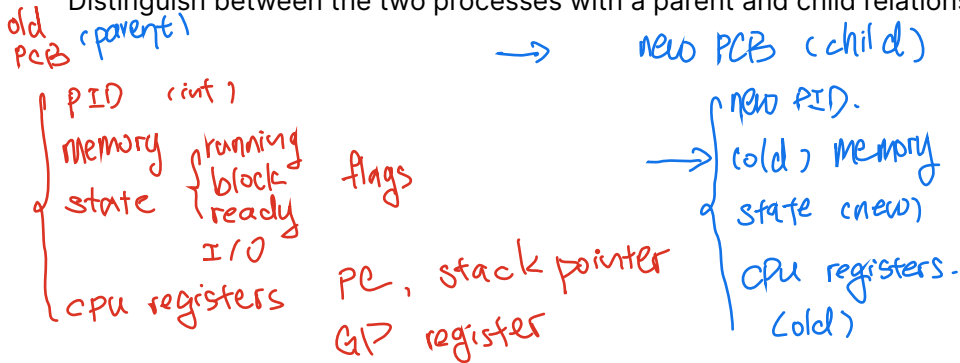
We create the process control block and load the program into memory
(this is what Windows does)

Unix decomposes process creation into more flexible abstractions

Instead of Creating a New Process, We Could Clone It

Pause the currently running process, and copy its PCB into a new one
This will reuse all of the information from the process, including variables!

Distinguish between the two processes with a parent and child relationship



fork Creates a New Process, A Copy of the Current One

`int fork(void)` as the following API:

- Returns the process ID of the newly created child process
 - 1: on failure
 - 0: in the child process
 - >0: in the parent process (child PID)

There are now 2 processes running

Note: they can access the same variables, but they're separate ←

Operating system does "copy on write" to maximize sharing

↓
page is only
copied on write

fork Examples

```
#include <stdio.h>
#include <unistd.h>
```

```
int main() {
    int value = 2;
    fork();
    printf("Hello world, %d\n", value);
    return 0;
}
```

PC →

P1. parent (PC)
P2. child (PC)

fork Examples

```
int main() {  
    int value = 2;  
    if (fork() == 0)   
        printf("Hello world\n");  
    else   
        printf("Hello world again\n");   
    return 0;  
}
```

fork Examples

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int value = 2;
    int pid = getpid(); ← PID current process.
    fork();
    value = 3;
    printf("Hello, world, %d, %lx!\n", value, (long int)&value);
    return 0;
}
```

execve Replaces the Process with Another Program, and Resets

execve has the following API:

- pathname: Full path of the program to load ←
- argv: Array of strings (array of characters), terminated by a null pointer
Represents arguments to the process
- envp: Same as argv
Represents the environment of the process
- Returns an error on failure, does not return if successful

execve-example.c Turns the Process into ls

```
int main(int argc, char *argv[]) {  
    printf("I'm going to become another process\n");  
    char *exec_argv[] = {"ls", NULL};  
    char *exec_envp[] = {NULL};  
    int exec_return = execve("/usr/bin/ls", exec_argv, exec_envp);  
    if (exec_return == -1) {  
        exec_return = errno;  
        perror("execve failed");  
        return exec_return;  
    }  
    printf("Hello world!\n");  
    return 0;  
}
```

main(argc, char *argv[])

./a.out

← argv

← environment

←

ls

execve('ls')

printf("hello world\n")

→ output: nothing.

Putting together: fork, wait and exec

On Linux, what happens when you run a command in the shell?

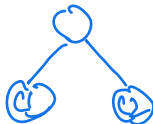
Let's try to run sleep 2

shell \$ sleep 2.
(PID 1100) $\rightarrow$ fork () [PID 1101]
 (child of PID 1100)
 child $\rightarrow$ execve ("sleep 2"). [PID 1101]

same PID as child



wait delays execution



wait uses the following API:

- status: Address to store the wait status of the process
- Returns the process ID of child process
 - 1: on failure
 - >0: the child with a change

The wait status contains a bunch of information, including the exit code

Use `man wait` to find all the macros to query wait status

You can use waitpid to wait on a specific child process

wait Example

```
int main() {  
    int pid = getpid(); ← parent PID_  
    fork();  
    2P → if (getpid() == pid) { ← parent  
        wait(NULL); ← child to exit  
        printf("Hello A\n");  
    }  
    else { ← child.  
        printf("Hello B\n");  
    }  
    return 0;  
}
```

Hello B
<child exit>
<wait returns>
Hello A

The Operating System Creates Processes

The operating system has to:

- Maintain process control blocks, including state
- Create new processes
- Load a program, and re-initialize a process with context

On Unix, the Kernel Launches A Single User Process

After the kernel initializes, it creates a single process from a program

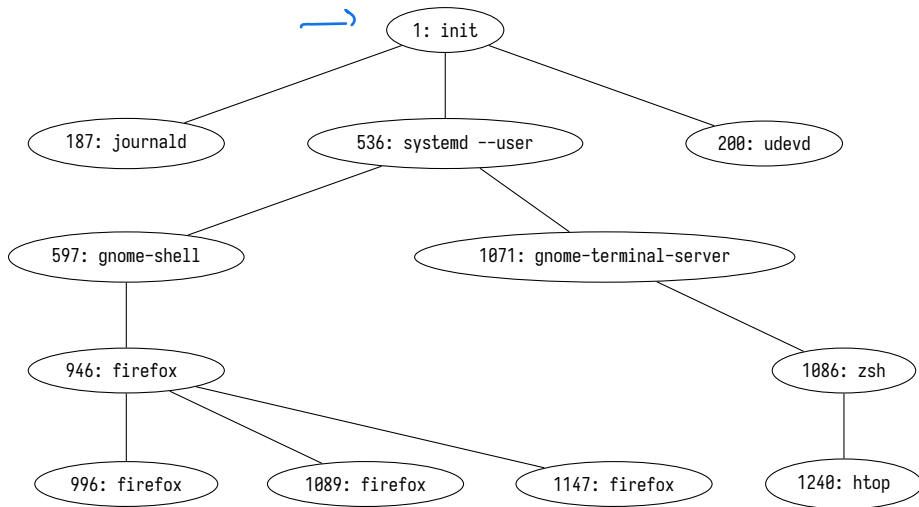
This process is called `init`, and it looks for it in `/sbin/init`

- Responsible for executing every other process on the machine

- Must always be active, if it exits the kernel thinks you're shutting down

For Linux, `init` will probably be `systemd` but there's other options

A Typical Process Tree on the Virtual Machine



How You Can See Your Process Tree

Use htop

You can press F5 to switch between tree and list view

Processes Are Assigned a Process ID (pid) On Creation

The process ID is just a number, and is unique for every **active** process

On most Linux systems the maximum pid 32768, and 0 is reserved (invalid)

Eventually the kernel will recycle a pid, after the process dies, for a new process

Remember: each process has its own address space (independent view of memory)

Maintaining the Parent/Child Relationship

Previously, we made sure that our parent exited last (by using sleep)

What happens if the parent process exits first, and no longer exists?

The Parent Process is Responsible for Its Child

The operating system sets the exit status when a process terminates (the process terminates by calling `exit`)

It can't remove its PCB yet ↩

The minimum acknowledgment the parent has to do is read the child's exit status

There's two situations:

- 1. The child exits first (zombie process)
- 2. The parent exits first (orphan process)

A Zombie Process Waits for Its Parent to Read Its Exit Status

The process is terminated, but it hasn't been acknowledged

A process may have an error in it, where it never reads the child's exit status

The operating system can interrupt the parent process to acknowledge the child

- It is just a suggestion and the parent is free to ignore it

- This is a basic form of IPC called a signal

The operating system has to keep a zombie process until it's acknowledged

- If the parent ignores it, the zombie process needs to wait to be re-parented

An Orphan Process Needs a New Parent

The child process lost its parent process

The child still needs a process to acknowledge its exit

The operating system re-parents the child process to init

The init process is now responsible to acknowledge the child

orphan-example.c The Parent Exits Before the Child

```
int main(int argc, char *argv[]) {  
    pid_t pid = fork();  
    if (pid == 0) { ← child  
        → printf("Child parent pid: %d\n", getppid());  
        sleep(2);  
        printf("Child parent pid (after sleep): %d\n", getppid());  
    }  
    else { ← parent.  
        sleep(1);  
    }  
    return 0;  
}
```

is : parent exits. ←
child sleeps 2s
 < >

zombie-example.c The Parent Monitors the Child To Check Its State

```
pid_t pid = fork();

if (pid == 0)
    sleep(2);

else {
    int ret;
    sleep(1);
    printf("Child process state: ");
    ret = print_state(pid);
    sleep(2);
    printf("Child process state: ");
    ret = print_state(pid);
}
```

You're Responsible for Managing Processes

The operating system maintains a strict parent/child relationship

You should be able to identify (and prevent) the following:

- ~~Zombie processes~~
- Orphan processes ←

Interprocess Communication (IPC)

Processes do not share any memory with each other

Some processes might want to work together for a task, so need to communicate information

IPC are mechanisms to share information between processes

Shared Memory

Signals are a Form of IPC that Interrupts

You could also press Ctrl+C to stop `./open-example`

This interrupts your programs execution and exits early

Kernel sends a number to your program indicating the type of signal

Kernel default handlers either ignore the signal or terminate your process

Ctrl+C sends SIGINT (interrupt from keyboard)

If the default handler occurs the exit code will be 128 + signal number

Pipe

A pipe is a half-duplex communication mechanism:

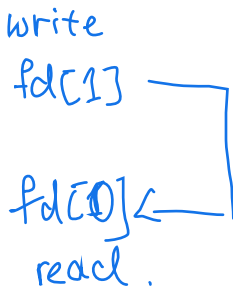
- Data written into ($fd[1]$) can be read from ($fd[0]$).
- Data flows in one direction only.

Regular pipes:

- Both file descriptors exist in the same process.
- The parent and child process share the pipe after a `fork()`.
- The parent writes to the pipe, and the child reads from it (or vice versa).

Named pipes (FIFOs):

- A named pipe allows communication between unrelated processes.
- Exists as a special file in the filesystem.



Pipe Example

```
int fd[2]; // File descriptors for the pipe
pipe(fd);

pid_t pid;
char write_msg[] = "Hello from parent!";
-> char read_msg[100];
pid = fork();

if (pid > 0) { ← parent
    write(fd[1], write_msg, strlen(write_msg) + 1);
} else { ← child
    read(fd[0], read_msg, sizeof(read_msg));
    printf("Child received: %s\n", read_msg);
}
```

Pipe Example

```
int fd[2]; // File descriptors for the pipe
pipe(fd);

pid_t pid;
char write_msg[] = "Hello from parent!";
char read_msg[100];
pid = fork();
if (pid > 0) {
    → close(fd[0]); ←
    write(fd[1], write_msg, strlen(write_msg) + 1);
    → close(fd[1]); ←
} else {
    close(fd[1]); ←
    read(fd[0], read_msg, sizeof(read_msg));
    close(fd[0]); ←
    printf("Child received: %s\n", read_msg);
}
```